LWYRD · MATCHING ALGORITHM · SUPPLEMENTARY REPORT

How to Make the Algorithm Meaningfully Stronger

A forward-looking roadmap covering new signals, data infrastructure, firm-side data gaps, weight calibration, and the path to ML — beyond the three specific bug fixes already implemented.

Companion to — Matching Algorithm Technical Report (June 2026)

Scope — Broader improvements: what to add, what to build, what to defer

Date — June 2026

CONTENTS

Table of Contents

CONTEXT

The Core Problem With the Algorithm Today

The matching algorithm is currently a **calibrated heuristic** — a set of hand-tuned rules that encode what the team believes a good match looks like. That is a reasonable starting point. The problem is that it has no mechanism to learn whether those beliefs are correct, and several of the signals it uses are either wrong (the three bugs now fixed) or too coarse to discriminate meaningfully between firms.

The result is a system where two firms with very different capabilities for a specific user can receive nearly identical scores, and where a genuinely strong match might score 71 while a weak one scores 68 — differences that are within the noise margin of the current weights.

Making the algorithm meaningfully stronger requires work at three levels:

- **Signal depth** — adding signals that the algorithm doesn't yet see, on both the user side and the firm side
- **Outcome feedback** — building the infrastructure to measure whether matches are actually working
- **Weight accuracy** — using outcome data to replace engineering guesses with evidence-based weights

> **WHAT THE THREE BUG FIXES ACCOMPLISHED**
>
> The fixes already implemented correct two cases where collected signals were silently discarded (language, billing model) and one systematic scoring error (individual-track industry bias). They make the algorithm *correct* about what it currently measures. The items in this report make it measure *more*, and eventually measure *better*.

PRIORITY 1 — MOST CRITICAL

Build an Outcome Feedback Loop

This is the most important single investment, and it should happen before anything else. Without it, every other improvement is unvalidatable. You can tune weights, add signals, and run experiments — but you cannot know if any of it made the matches better, because "better" has no definition without outcome data.

What "outcome" means in this context

There are three tiers of outcome signal, in order of ease to collect and strength of signal:

Event	How to capture	Signal strength
Contact click	Log a `match_event` when the user clicks "View Profile" or a contact link from a match card	Weak — interest, not engagement
Firm contacted	Track outbound contact link clicks (email, phone, website) on the firm profile page with a `match_id` parameter	Medium — strong intent signal
Engagement confirmed	Post-match email after 2 weeks: "Did you hire a firm?" — or CRM integration if firms report back	Strong — ground truth

What to build

One new database table and one server action are all that's needed to start collecting contact events:

```
-- New table
create table match_events (
  id           uuid primary key default gen_random_uuid(),
  match_id     uuid references matches(id) on delete cascade,
  user_id      uuid references auth.users(id),
  event_type   text not null,   -- 'view_profile' | 'contact_click' | 'engaged'
  occurred_at  timestamptz default now()
);

-- RLS: users can insert their own events, admins can read all
alter table match_events enable row level security;
```

Log a `view_profile` event whenever a user navigates from a match card to `/firms/[id]` with a match context. Log a `contact_click` event from the firm profile page when the user clicks the contact button. The `match_id` is already stored in sessionStorage (`lwyrd_match_scores`) and can be passed as a query param or resolved server-side from the user's most recent submission.

> **WHY THIS COMES FIRST**
>
> Every other priority in this document — weight calibration, ML, A/B testing — requires outcome labels to validate. Start collecting now. Even six months of contact-click data is enough to validate whether the current weight ordering (budget > quality > industry > stage >...) is actually correct.

PRIORITY 2 — HIGHEST SIGNAL VALUE

Add Sub-Matter Type Matching

The algorithm currently treats an entire practice area as one undifferentiated pool. "Dispute resolution" includes commercial litigation, employment arbitration, securities disputes, IP infringement, and consumer claims. A firm specialising in securities arbitration and a firm specialising in employment class actions are indistinguishable to the algorithm. This is the largest source of ranking noise currently present.

What the V2 intake already collects

The V2 intake wizard already asks sub-matter questions — stored as `sub_question_json` per submission. For example, a dispute-resolution intake might capture whether the matter is commercial, employment-related, or IP-related. This data is being stored but never read by the matching function.

What's missing on the firm side

Firms have no structured sub-matter expertise tags. To close this gap, one new join table is needed:

```
create table firm_sub_matter_expertise (
  id                uuid primary key default gen_random_uuid(),
  firm_id           uuid references firms(id) on delete cascade,
  practice_area_slug text not null,
  sub_matter_slug   text not null,         -- e.g. 'commercial-litigation', 'securi
  is_primary        boolean default false,  -- true if this is the firm's dominant
  unique (firm_id, practice_area_slug, sub_matter_slug)
);
```

This table would be populated through the admin panel during firm onboarding. An admin tagging step of ~2 minutes per firm is sufficient.

The matching change

A new `scoreSubMatter()` function (~8 pts weight) would read the user's sub-matter answer from the intake and compare it against the firm's `firm_sub_matter_expertise` rows for the matching practice area. An exact sub-matter match gets full points; a practice-area match with no sub-matter data gets a neutral score; a sub-matter mismatch gets a low score. The function would be called from `matchFirms()` and automatically handled by the existing `(totalPts / maxPts) × 100` normalisation.

THIS REQUIRES A SCHEMA MIGRATION

Unlike the three bug fixes, this improvement requires creating the `firm_sub_matter_expertise` table, adding admin UI to tag firms, and a seeding pass across all existing firms. Estimated effort: 1–2 days including firm tagging.

PRIORITY 3

Score the Intake Signals Already Being Collected

The three bug fixes already implemented addressed language preference, billing model, and individual-track industry bias. But the intake collects additional signals that still go unused in matching. These are the lowest-effort improvements remaining because the user-side data already exists.

A Score prior counsel experience

No schema change ~15 lines in matching.ts

All three tracks ask whether the user has previously worked with a law firm (`sf6 / if2 / bf6`). This is stored per submission but not scored. It is a meaningful proxy for the user's sophistication level:

- **First-time users** benefit from firm sizes that offer more hand-holding (boutique or mid-size with high partner involvement)
- **Repeat users** may prefer efficiency-focused firms and are more comfortable with associate-heavy work

A `scorePriorCounsel()` function could use this signal to modulate the firm-size score, or as an independent 4-pt signal.

B Score matter complexity indicators

No schema change | ~20 lines in matching.ts

The intake already collects signals that proxy matter complexity but are currently unused in matching:

- **Transaction size** (`b4m` , `s5h`) — a $50M M&A transaction needs a larger firm than a $1M deal
- **Employee count** (`b2` , `s5j`) — employment matters at 200-person companies need different expertise than at 10-person companies
- **Founder/party count** (`s4`) — multi-founder situations add complexity to equity and governance work

Complexity signals should influence the firm-size score (complex matters benefit from larger firms) rather than being scored independently.

C Score entity type for corporate/small-business tracks

No schema change | ~12 lines in matching.ts

The small-business track collects entity type (`b4b`): LLC, C-corp, S-corp, partnership, sole proprietorship. Some firms specialise in C-corp / venture-backed structures; others are generalists. If the `Firm` object carries entity type expertise (currently it doesn't, but it could be added to firm attributes without a migration), this becomes a direct scoring signal. Minimum viable version: weight C-corp formation intake answers toward larger firms that typically handle VC-backed entities.

PRIORITY 4

Deepen Firm-Side Data

The algorithm can only score what it knows about each firm. Several firm attributes are either absent, stored at insufficient granularity, or populated inconsistently across the database. These gaps limit the ceiling on matching accuracy regardless of how many intake signals are added.

Gap	Current state	What's needed
Licensed jurisdictions	All firms assumed NY-licensed. Out-of-state capability inferred from HQ location string parsing — fragile.	`firm_licensed_states: text[]` column. Firms licensed in CA, TX, DE, etc. should be tagged explicitly. Removes the HQ-parsing heuristic entirely.
Sub-matter expertise	Not represented at all. Firm's practice area is a flat array of slugs.	`firm_sub_matter_expertise` join table (see Priority 2). Most important firm-side data gap.
Language coverage	`languages: string[]` field exists on Firm, but population is inconsistent — many firms have an empty array even though their attorneys speak multiple languages.	Require language field during firm onboarding. Add to admin form validation. Run a backfill pass on existing firms.
Billing model granularity	Single `billingModel` value per firm. A firm that offers both hourly and flat-fee is forced to pick one.	`billing_models: text[]` — array of offered models. One-line schema migration; the scoring function checks `.includes(pref)` instead of `=== pref`.
Practice area attorney depth	Firm size (boutique / mid-size / large) is a rough proxy for capacity. No data on how many attorneys work in a specific practice area.	Add `attorney_count` to `firm_practice_areas` join table. A "large" firm with 2 IP attorneys is worse for an IP matter than a boutique with 8. This is the highest-value firm attribute to add after sub-matter.
Assessment data freshness	LWYRD assessment scores are static snapshots with no	Add `assessed_at` timestamp to assessment records.

Gap	Current state	What's needed
	date. A firm assessed 18 months ago may have changed significantly.	Implement a staleness flag (e.g., >12 months old). Surface staleness in admin panel. Penalise stale assessments slightly in `scoreQuality()`.

HIGHEST IMPACT FIRM-SIDE IMPROVEMENT

Language coverage backfill and billing model array migration are the cheapest wins — each is one admin pass plus a small schema change. Sub-matter tagging and licensed-states are the most impactful but require more onboarding investment.

PRIORITY 5 — AFTER OUTCOMES ARE COLLECTED

Calibrate Weights Against Outcomes

The current weights — 24 pts for budget, 22 for quality, 18 for industry, etc. — were set by engineering judgement. They represent a reasonable hypothesis about what makes a good match, but they are not validated against data. This matters because wrong weights can invert rankings: if budget is actually less predictive of contact than industry fit, the current weighting is actively hurting result quality.

What calibration requires

You need two things: outcome labels (from Priority 1) and enough volume. As a rough threshold: 300–500 labelled contact events across a reasonably diverse set of intake types. Below that, any weight changes you make are just swapping one guess for another.

The simplest valid calibration approach

Once you have outcome labels, the simplest statistically valid approach is **logistic regression**. For each submission-firm pair, you have: the intake signal values, the match score components, and a binary outcome (contacted / not contacted). Fit a logistic regression model with the per-signal point totals as features and the contact event as the label. The learned coefficients become your new weights.

```
# Pseudocode — run once you have ~400 contact events
from sklearn.linear_model import LogisticRegression

X = [[budget_pts, quality_pts, industry_pts, stage_pts,
      size_pts, location_pts, timeline_pts, billing_pts, lang_pts]
     for each match_firm_pair]

y = [1 if contact_event exists else 0
     for each match_firm_pair]

model = LogisticRegression().fit(X, y)
new_weights = model.coef_[0]  # replace hardcoded MAX values
```

This approach is interpretable, fast, requires no ML infrastructure, and produces weights that can be read back into the existing scoring functions directly. It is the right first step before any more sophisticated model.

> **DO NOT RECALIBRATE FROM SUBMISSION COUNT ALONE**

The number of intake submissions is not a useful proxy for readiness to recalibrate. You need contact events — evidence that a user expressed real intent toward a specific firm. Without contact events, recalibration is just guessing with extra steps.

PRIORITY 6

Fix Score Reliability at Thin Pool Sizes

When the matching pool shrinks to 2–4 firms (due to hard filters or niche practice areas), the current algorithm produces scores that look precise but are not. A rank-1 score of 74 and a rank-2 score of 71 suggest meaningful differentiation — but with only 3 firms in the pool, a 3-point difference is well within the noise of the current weight estimates.

Two complementary fixes address this:

1. Surface the pool size in the UI

Pass `poolSize` as a field in the `MatchResult` type. Display a soft notice on the results page when `poolSize < 5`: "We found only a few firms matching your criteria — consider adjusting your state, budget, or firm size preference to see more options." This manages user expectations without changing any scoring logic.

2. Widen geographic coverage instead of softening filters

The root cause of thin pools for non-NY users is that the firm database is almost entirely NY-licensed. The correct fix is to onboard firms licensed in other high-demand states (CA, TX, FL, IL) rather than softening the hard filters. Softening filters would show users firms that genuinely cannot serve them, which is worse than showing fewer results.

3. Consider replacing point scores with match tiers for display

An alternative to displaying a raw integer score is a tier label: **Strong match** (score ≥ 75), **Good match** (score 55–74), **Possible match** (score < 55). Tiers are less gameable, more honest about precision, and easier for users to interpret. The underlying integer score remains for ranking and analytics — only the display changes.

> **WHY THIS MATTERS FOR RETENTION**
>
> A user who sees a "74 match score" next to a firm and then has a poor experience is confused — the number suggested precision. A user who sees "Strong match" with a

clear rationale has better calibrated expectations. Tier labels reduce the risk of users blaming LWYRD for a score that looked authoritative but wasn't.

PRIORITY 7 — LONG-TERM

The Machine Learning Path — When and How

Machine learning is the right long-term architecture for a matching system at meaningful scale. It is not the right approach right now, and attempting it prematurely will cost engineering time without improving results. This section describes the correct conditions and method.

When ML becomes viable

Condition	Threshold	Status
Labelled outcome events	≥ 1,000 contact events across diverse intake types	Currently: 0 contact events collected
Outcome diversity	Outcomes distributed across multiple practice areas and firm types — not dominated by one category	Depends on outcome collection
Feature stability	Intake signals and firm attributes are stable (not changing every sprint)	Partially met — V2 intake is stable
Retraining infrastructure	A process to retrain and deploy updated weights without a code deploy	Not built

The right model type

This is a **Learning-to-Rank** problem: given a user and a set of candidate firms, rank the firms by predicted engagement likelihood. The correct model family is **gradient-boosted trees trained with a ranking loss**.

- **LightGBM with LambdaRank loss** — fast, interpretable feature importances, works well at modest data volumes (<100K training pairs)
- **Features** — all current signal values (per-criterion point totals) plus new signals from Priorities 2–4
- **Label** — binary (1 = contact event occurred within 14 days, 0 = no contact)
- **Evaluation metric** — NDCG@3 (normalized discounted cumulative gain at 3 results, since users rarely look past the third card)

Deployment

The cleanest deployment architecture for a Next.js / Supabase stack at current scale is to store trained model weights as a JSON blob in a `model_versions` table, load them at server startup, and run inference in the existing `matchFirms()` function by replacing the hardcoded per-signal weights with the loaded coefficients. No separate ML serving infrastructure is needed at this scale.

> **DO NOT ATTEMPT ML BEFORE 1,000 LABELLED EVENTS**
>
> With fewer labels, any ML model will overfit to noise in the training data and perform worse than the current weighted heuristic. The logistic regression calibration approach from Priority 5 is the correct intermediate step.

GUARDRAILS

What Not to Do

Tempting path	Why to avoid it
Train an ML model before collecting outcome data	No ground-truth labels → model trains on noise → worse rankings than the current heuristic, with more engineering complexity.
Soften the hard filter thresholds to increase pool sizes	This shows users firms that genuinely cannot serve them. Better to show 2 accurate results than 6 inaccurate ones. Grow the firm database instead.
Add more intake questions to collect signals	Intake completion rate drops with each additional question. The intake already collects signals that aren't being scored (prior counsel, entity type, complexity). Score existing signals before adding new questions.
Recalibrate weights manually without data	The current weights are already the result of engineering judgement. A second round of guessing doesn't make them more accurate — it just makes a different guess. Wait for outcome data.
Display high-precision scores (e.g., "87.4") to users	The algorithm's margin of error on individual scores is several points. Decimal precision implies accuracy that doesn't exist and erodes trust when users have a bad experience with a "87.4 match."
Build A/B testing before having a metric to optimize	A/B testing requires a north-star metric (e.g., contact rate per result page view). Without outcome tracking, there is no metric to optimise and A/B tests have no valid interpretation.

SEQUENCING

Suggested Sequencing

This is a rough ordering based on impact and dependency. Items in each phase are independent within the phase.

- **Done — Bug fixes (matching.ts only, no schema)**

  Score language preference · Fix individual-track industry bias · Score billing model preference

- **Phase 1 — Infrastructure foundation (no ML, minimal schema)**

  Build `match_events` table and log contact clicks · Backfill `languages[]` on all firms · Migrate `billingModel` to `billing_models[]` array · Surface pool size and thin-pool notice in results UI

- **Phase 2 — Signal expansion (schema migrations + admin work)**

  Create `firm_sub_matter_expertise` table · Tag all existing firms with sub-matter types · Add `firm_licensed_states[]` column · Replace HQ-string location heuristic with licensed-state check · Add `attorney_count` to `firm_practice_areas` · Score prior counsel signal · Score matter complexity

- **Phase 3 — Weight calibration (after ~400 contact events)**

  Export match signal data + contact events · Run logistic regression · Replace hardcoded MAX weights with calibrated coefficients · Validate on held-out submissions

- **Phase 4 — Consider match tier display (UX change)**

  Replace integer score display with Strong / Good / Possible tiers · Keep raw score for admin analytics · User-test both approaches

- **Phase 5 — ML ranking model (after ~1,000 labelled events)**

  Train LightGBM LambdaRank on contact events · Store weights in `model_versions` table · Load at server startup in `matchFirms()` · Monitor NDCG@3